(c) The hash value is calculated from the key field using the formula: `Key MOD 200`

The function `CalculateHash()` takes a key field as a parameter and calculates and returns the hash value for the key field.

Write program code for `CalculateHash()`

Save your program. Copy and paste the program code into part **2(c)** in the evidence document.

[2]

(d) The procedure `InsertIntoHash()`:

- takes a record of type `NewRecord` as a parameter
- uses `CalculateHash()` to calculate the hash value for the key field in the record
- checks if the hash value index in `HashTable` currently stores an empty record
  - if the index stores an empty record, store the parameter in this index
  - if the index does **not** store an empty record, store the parameter in `Spare`

You can assume there will always be enough space in `Spare` to store any collisions.

Write program code for `InsertIntoHash()`

Save your program. Copy and paste the program code into part **2(d)** in the evidence document.

[6]

(e) The text file `HashData.txt` stores up to 200 rows of data to be stored into the hash table. Each row contains three integer numbers separated by commas.

The first number is the key field, the second number is item 1 and the third number is item 2.

For example:

The first row in the text file contains: `646`, `12`, `568`

The key field is `646`, item 1 is `12` and item 2 is `568`

The procedure `CreateHashTable()`:

- opens the file `HashData.txt`
- creates a record for each row of data in the file
- calls `InsertIntoHash()` with each record.

Write program code for `CreateHashTable()`

Save your program. Copy and paste the program code into part **2(e)** in the evidence document.

[5]